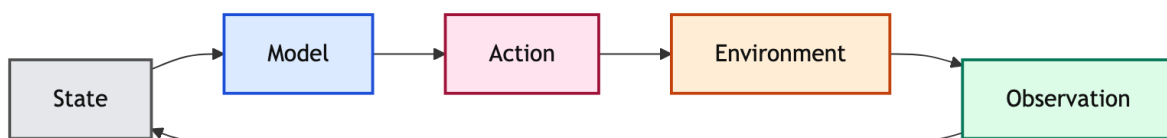


第 05 讲：Agent Harness 的基本模式

Coding Agent Notes (June 2026)

1 核心图景

一个 coding agent 不是“模型加一个 prompt”。真正让模型变成软件工程 worker 的，是外面那层 harness：它决定模型看见什么，能做什么，动作在哪里运行，反馈怎样返回，历史如何保存，什么时候停止，以及最后的结果怎样验证。



这个闭环可以读成一句话：**状态进入模型，模型产生动作，动作改变环境，环境返回观察，观察再更新状态**。只要这个循环存在，就已经有了 agent harness 的基本形状。不同系统的差别，主要体现在每个节点怎么设计。

Coding agent harness = state + action space + runtime environment + observation formatting
+ trajectory memory + stop policy + validation.

这一定义有一个直接后果：harness 不是透明包装。它会改变同一个模型的实际能力。一个模型在 shell-only 环境里可能反复犯编辑错误；换成有 file viewer、edit command 和 lint feedback 的 ACI 后，可能更稳定。同一个模型如果被放进 Docker sandbox、给出测试反馈、保留清晰轨迹，也会比只生成一次 patch 更像软件工程 agent。

1.1 两层状态

理解 harness 时，最容易混淆的是“模型上下文”和“真实环境状态”。模型上下文是 token 序列：任务、历史、文件片段、日志摘要、系统提示。真实环境状态是文件系统、进程、依赖、数据库、浏览器页面、git 工作区、测试缓存。agent 的每一步，其实都在让这两层状态互相校准。

如果模型上下文说“我已经改好了文件”，但工作区没有对应 diff，这就是状态不一致。如果环境里测试失败，但 observation 被截断到看不见 traceback，模型也会形成错误判断。长期任务最难的地方，不是让模型多想一步，而是让这两层状态在几十步之后仍然对齐。

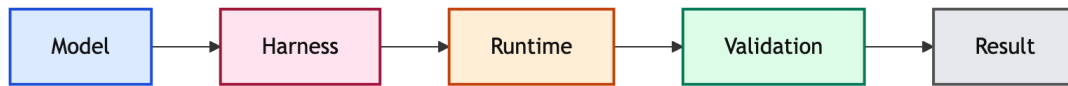
Harness 的核心责任：

让 token state 足够代表 execution state，
再让每次 action 把 execution state 的变化稳定反馈回 token state。

这个视角也解释了为什么同一个模型在不同 agent 系统里差异很大。模型本身的推理能力没有变，但它能否看到关键状态、能否安全改变环境、能否从反馈里修正错误，全都由 harness 决定。

2 为什么 harness 不是 wrapper

wrapper 只是把输入传给模型、把输出拿回来。harness 则是一个控制系统。它让文本输出变成可执行动作，又把外部世界的结果压缩回模型能理解的观察。



它定义 action space。

模型不能直接“改代码”。它必须通过某种动作表示：bash 命令、编辑命令、Python code action、浏览器动作、patch diff，或者固定 pipeline 中的某个 stage。动作空间越合适，模型越容易把意图转化为稳定操作。

它定义 runtime。

动作需要运行在某个地方。本地 shell、Docker、远程 sandbox、浏览器、IPython server、CI 环境，都会影响可见状态和失败类型。runtime 不只是安全边界，也是能力边界。

它定义 observation。

命令输出、错误栈、return code、文件 diff、lint message、测试日志都可能成为 observation。观察太少，模型不知道发生了什么；观察太长，模型被噪声淹没；观察格式不稳定，模型难以形成可靠策略。

它定义 memory。

长期任务需要记住做过什么。最简单的是线性 messages；复杂平台会维护 event stream、cost、metadata、delegation state。memory 决定 agent 是否能复盘、恢复、调试和训练。

它定义 stop 和 validation。

agent 必须知道什么时候结束，系统也必须知道结果是否可交付。step limit、cost limit、wall-time limit、submit command、patch apply、tests、regression checks，都属于 harness。

2.1 三条边界

把 harness 当控制系统后，可以把它的设计拆成三条边界。

语义边界 决定模型读到的任务如何被表达。issue 原文、失败测试、相关文件、repo map、用户提示、历史摘要，都属于这一层。语义边界的失败通常表现为模型“方向错”：修了无关文件、误解需求、把 benchmark trick 当成真实需求。

执行边界 决定模型动作能影响什么。它包含 shell 权限、容器文件系统、网络、git 状态、超时、资源配额。执行边界的失败通常表现为模型“做不到”：命令被拒绝、依赖缺失、测试不可运行、环境污染。

验证边界 决定什么算完成。它包含 submit 规则、patch 格式、测试选择、回归检查、人工确认。验证边界的失败通常表现为模型“看似完成”：局部测试通过，但真实 issue 没修；新测试通过，但旧行为被破坏。

Insight. 模型能力只有通过 harness 才能进入真实环境。harness 设计得差，强模型会被浪费在坏动作、坏观察和坏状态里；harness 设计得好，模型能力才会稳定转化为工作。

3 最小 loop: mini-SWE-agent

mini-SWE-agent 是理解 harness 的好起点，因为它刻意把系统做得很小。它的核心 loop 可以概括为：初始化 messages，调用模型，执行 actions，把输出格式化为 observation，追加到 messages，然后重复，直到出现 exit。

从源码看，DefaultAgent.step() 基本就是：

步骤	mini-SWE-agent 概念	harness 概念
初始化 system/user message	task + templates	初始 state
query()	model.query(messages)	policy/action proposal
execute_actions()	env.execute(action)	action execution
format observations	model-specific formatting	observation channel
append messages	linear history	trajectory memory
exit role / Submitted	stop condition	submission policy

这个极简 loop 的价值在于，它把 harness 的本质暴露得很干净。没有复杂工具，没有大型 UI，没有多 agent，也没有专门 file editor；但核心元素都在：状态、动作、环境、观察、预算、退出、轨迹保存。

mini-SWE-agent 的 LocalEnvironment 用 subprocess.run 执行命令，返回 stdout、returncode 和异常信息。这个设计看似朴素，但有一个重要性质：每个动作独立执行，不保留 stateful shell session。这使得 sandbox 替换更容易，也让轨迹更容易解释。

3.1 一次 repair turn 里发生了什么

假设任务是修一个 repo bug。最小 loop 的一次迭代可以读成四个动作。

第一，模型提出可执行假设。它可能决定先运行测试、搜索函数名、打开某个文件，或直接修改代码。这一步不是最终答案，而是对环境的一次探测。

第二，harness 把文本动作交给环境。如果动作是 shell command，环境负责切换到正确目录、设置超时、捕获输出、记录异常。如果动作是结构化 tool call，harness 还要解析参数、校验 schema、处理无效调用。

第三，环境产生可观察结果。结果可能是测试失败、文件片段、空输出、权限错误、timeout、安装日志。原始结果通常不能直接塞回模型，需要裁剪和格式化。

第四，harness 更新 trajectory。这一步决定下一轮模型能否知道“刚才做过什么”。如果历史只保留自然语言总结，可能丢掉命令细节；如果保留全部日志，可能挤掉任务核心上下文。

这四步合起来，才是“agent 做了一步”。模型输出只是其中一部分；没有执行、观察和状态更新，它只是生成了一段文本。

3.2 极简不等于粗糙

极简 harness 的重点不是“工具少所以能力弱”，而是“控制流简单所以容易理解和复现”。bash-only 设计让任何模型都能运行，不依赖特定 tool-calling schema；线性 history 让轨迹和模型输入几乎一致；独立命令让本地、Docker、Singularity、远程执行后端更容易替换。

这对研究尤其重要。如果目标是比较模型能力，过强或过复杂的 scaffold 可能掩盖模型差异；如果目标是训练 agent，过多隐式工具可能让轨迹难以学习。一个小 harness 可以作为 baseline，让我们看清模型到底承担了多少工作。

4 State 与 trajectory

state 是 agent 当前知道的一切。它不只是 prompt 文本，也包括任务描述、当前目录、历史动作、历史观察、成本、剩余步数、当前文件状态、用户反馈、甚至委托给其他 agent 的子任务状态。

trajectory 是 state 随时间展开后的记录。它回答：模型看到了什么，做了什么，环境返回了什么，为什么停止。没有 trajectory，就很难解释一个 agent 为什么成功或失败。



mini-SWE-agent 使用线性 messages。每次模型输出和环境观察都追加进去。OpenHands 则把 action 和 observation 存进 event stream，并把 cost、metadata、delegation 等信息放进 state。两者复杂度不同，但核心作用一致：让下一步决策有上下文，让事后分析有证据。

表示方式	优点	代价
线性 messages	简单、透明、适合调试和微调	长任务容易累积噪声
event stream	结构化、可视化、可 replay	平台实现更复杂
压缩 history	节省上下文、延长任务	可能丢掉细节
外部日志	便于审计和 QA	模型未必能直接利用

trajectory 的重要性还在于训练。可执行任务训练和 RL 都需要知道每一步发生了什么，哪些 action 有效，哪些 observation 导致了修正。一个只保存最终 patch 的系统，很难变成高质量训练数据；一个保存完整轨迹的系统，才能分析 failure mode、过滤坏样本、构造偏好数据。

4.1 state 设计的两个坑

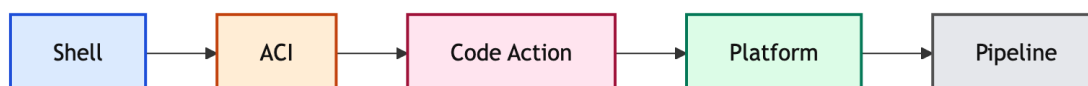
第一个坑是把 state 当作“越多越好”。长上下文确实能容纳更多文件和历史，但 agent 需要的是可行的状态，而不是完整转储。把整个测试日志、整个仓库文件、所有历史命令都塞进去，常常会让模型找不到当前最重要的差异。

第二个坑是把 state 当作“只给当前一步需要的信息”。软件工程任务经常需要回看：之前哪个测试失败过、哪个文件已经改过、为什么放弃某条路线、用户有没有约束不要改 public API。过度压缩会让 agent 重复探索，甚至撤销自己刚刚做对的修改。

所以 state management 的目标不是最大化上下文，也不是最小化上下文，而是维护一个可追踪的工作记忆：当前目标、已知事实、已尝试动作、关键失败、当前 diff、剩余风险。OpenHands 的 event stream 和 mini-SWE-agent 的 linear history，是复杂度不同的两种答案。

5 Action space: agent 能做什么

Action space 是 harness 最关键的设计之一。它决定模型把意图转化为外部动作的接口。



5.1 Shell-only

shell-only 把 bash 当作唯一工具。优点是通用、模型熟悉、实现简单；缺点是很多软件工程动作不够稳定。比如多行编辑、查看局部文件、解释 silent command、处理交互式命令，都可能让模型犯错。mini-SWE-agent 的选择是把复杂度尽量交给模型和 shell。这使它非常适合作为 baseline：如果一个模型在 bash-only harness 里能完成任务，说明它本身有较强环境操作能力。

5.2 Agent-Computer Interface

SWE-agent 的 ACI 思路是：不要把人类 shell 原样交给模型，而是给模型更友好的动作。比如 search/navigation、file viewer、edit command、lint feedback、history compression。ACI 的目标不是增加花哨工具，而是减少模型在低层操作上的无谓失败。

这里的核心原则是：动作应该简单、紧凑、反馈明确。一个好的 edit action 可以把“打开文件、定位行号、替换多行、查看结果、发现语法错误”合并成一个稳定动作。

5.3 Code Action

CodeAct 提出用可执行 Python code 作为统一 action space。相比 JSON/tool call，code action 允许模型组合多个工具，利用 Python 包，写控制流，并根据执行结果 self-debug。它适合那些工具之间有数据流、需要临时计算、需要组合操作的任务。

对 coding agent 来说，code-as-action 的意义在于：模型原本就擅长写代码，把 action 写成代码可以复用它的预训练能力。但它也要求 sandbox 更可靠，因为可执行代码的能力越大，风险越大。

5.4 Platform Actions

OpenHands 代表平台化 action space：bash、IPython、browser、message、delegation、skills 都通过统一 event stream 与 runtime 交互。平台 action 的优势是可扩展、可 UI 可视化、可支持多任务和多 agent；代价是系统复杂度上升。

5.5 Pipeline

Agentless 提醒我们：harness 不一定要让模型自由选择下一步。它可以把 policy 固定成 localization、repair、validation 三阶段，让 LLM 只在每个阶段承担子任务。这种 pipeline harness 牺牲了一部分自治性，但换来可控性、低成本和更容易 debug 的流程。

Action style	适合场景	主要风险
Shell	通用本地操作、baseline	编辑脆弱、输出噪声、命令失败
ACI	repo repair、稳定编辑	工具设计可能过拟合任务
Code action	组合工具、数据流、自调试	sandbox 风险、代码执行副作用
Platform	产品/研究平台、多任务	架构复杂、调试面变大
Pipeline	可控修复流程、批量评测	灵活性不足、stage 误差传播

5.6 动作粒度：太粗和太细都危险

Action space 的核心问题不是“工具多不多”，而是动作粒度是否匹配任务。

动作太细，模型要承担大量低层 bookkeeping。比如只给裸 shell，模型需要自己处理搜索、分页、行号、编辑、验证、重试。强模型可以做到，但失败会变得琐碎：引号没闭合、sed 命令写错、行号漂

移、编辑后没检查。

动作太粗，harness 可能替模型做掉关键决策。比如一个“自动修复 issue”的黑盒工具如果直接返回 patch，模型就很难学习中间的定位、假设、验证过程；系统也很难解释失败来自定位还是修复。

好的动作粒度通常满足三个条件：**可组合**，模型能把多个小动作拼成策略；**可观察**，每个动作后能得到有用反馈；**可恢复**，动作失败后还有退路。SWE-agent 的 ACI、CodeAct 的 code action、Agentless 的分阶段 pipeline，都可以从这三个条件来比较。

5.7 可逆性与审计

软件工动作还需要考虑可逆性。读文件、搜索、运行测试几乎没有副作用；编辑文件、安装依赖、启动服务有副作用；删除文件、推送代码、修改远程资源则风险更高。一个成熟 harness 会把动作按风险分层，并把高风险动作放在更强的 sandbox、确认或审计之下。

这也是为什么 trajectory 不能只保存“模型想法”。真正需要审计的是动作和环境变化：执行了什么命令、改了哪些文件、测试结果是什么、提交前 diff 如何。对产品级 agent 来说，审计不是额外功能，而是 trust 的基础。

6 Observation：反馈如何回到模型

Observation 是 action 的结果，也是下一步决策的证据。对 coding agent 来说，观察通常来自命令输出、测试日志、文件内容、diff、浏览器状态、lint error、return code 或环境异常。

观察设计的难点在于平衡信息量。反馈太短，模型不知道怎么修；反馈太长，模型抓不住重点。SWE-agent 强调 feedback should be informative but concise，OpenHands 把每次 action execution 的结果变成 observation 写回 event stream，mini-SWE-agent 则直接返回 output、returncode 和 exception 信息。

Observation 类型	好处	常见问题
stdout/stderr	原始、完整	长日志噪声大
return code	简洁判断成功/失败	不解释失败原因
post-edit view	让模型看到修改效果	需要稳定行号/上下文
lint/test error	指向可修复问题	可能过长或误导
collapsed history	节省上下文	可能丢关键细节

一个高质量 harness 会把 observation 处理成“足够行动”的形式。比如 edit 后显示修改后的局部文件，让模型立刻发现缩进或括号错误；测试失败时保留关键 traceback，而不是把几千行日志全部塞回；旧观察可以压缩，但要保留动作顺序和关键信号。

Insight. Observation 不是日志转发，而是反馈设计。它决定模型能否把环境失败转化为下一步修正。

6.1 从 raw log 到 actionable feedback

测试日志本身不是高质量 observation。高质量 observation 至少回答三个问题：这一步有没有成功；如果失败，失败在哪里；下一步最可能查什么。

例如，一个 pytest 失败日志可以被拆成：失败测试名、异常类型、关键 traceback、actual/expected 差异、相关文件行号、命令 return code。保留这些信息，模型通常就能继续定位；保留完整环境安装日志，反而会冲淡主要信号。

编辑反馈也是同理。只告诉模型“edit succeeded”是不够的，因为它不知道实际文件是否符合预期；直接展示整个文件又可能太长。更好的方式是展示修改后的局部窗口、diff 或 lint feedback，让模型把“我以为我改了什么”和“文件实际变成什么”对齐。

6.2 历史压缩不是总结作文

当任务变长，history compression 会变得必要。但压缩不能只写成漂亮总结。对 harness 来说，压缩后的历史必须保留可操作事实：已经运行过哪些测试、哪些失败仍未解决、哪些文件被改动、哪些假设被排除、当前 patch 的风险点在哪里。

如果压缩只保留“我已经尝试修复并发现问题可能在 parser”，模型下一步仍然不知道证据来自哪里。更好的压缩应当接近工程 notebook：命令、结果、结论、下一步。这样压缩后的 state 才能继续驱动行动。

7 Runtime 与 sandbox

runtime 是动作真正发生的地方。没有 runtime，agent 只是生成文本；有了 runtime，agent 才能运行测试、修改文件、下载依赖、查看错误、验证 patch。

mini-SWE-agent 提供 local、Docker、Singularity、SWE-ReX、bubblewrap、contree 等环境选择。OpenHands 使用 Docker sandbox 和 action execution API，把 bash、Python、browser 等动作统一转成 observations。SWE-agent 也强调通过 Docker 等隔离环境降低执行风险。

Runtime	优点	风险 / 代价
Local	快、简单、贴近用户环境	安全弱、依赖污染
Docker	隔离、可复现、适合评测	构建和启动成本
Remote sandbox	易扩展、适合批量任务	延迟、资源管理、调试复杂
Browser runtime	支持 UI/E2E 任务	observation 更难压缩
CI/runtime	接近交付验证	慢、环境 flaky

runtime 既是安全问题，也是能力问题。一个没有测试环境的 agent 只能猜测 patch 是否正确；一个能运行测试但日志处理糟糕的 agent，会被反馈噪声拖慢；一个隔离性不足的 agent，可能破坏用户文件或泄露凭据。好的 harness 要同时考虑能力、成本和 containment。

7.1 可复现环境是 agent 能力的一部分

很多 repo issue repair 失败，并不是模型不会写 patch，而是环境不可复现：依赖版本不对、测试数据缺失、系统包不同、命令需要隐含服务、网络不可用。对人类工程师来说，这些是环境问题；对 agent 来说，它们会直接变成能力上限。

因此，runtime 设计要尽量把“任务本身难”和“环境启动难”分开。Docker image、lockfile、setup script、缓存策略、超时策略、测试选择，都在帮助 harness 降低环境噪声。一个 clean benchmark instance 应该让 agent 的主要挑战落在理解和修改代码上，而不是猜测如何启动项目。

7.2 隔离不是只是为了安全

Sandbox 常被理解为防止危险命令，但它还有两个更工程化的作用。

第一，隔离让结果可复现。同一个任务在同一个 image 中运行，才便于比较模型、比较 harness、复盘失败。第二，隔离让失败可丢弃。agent 走错路线后，可以重置环境、重新 apply patch、重新运行

验证，而不必担心本地状态已经被污染。

这解释了为什么 OpenHands 这类平台会把 runtime 做成核心组件，而不是简单调用用户机器上的 shell。平台要支持长任务、多任务、人机协作和评测，就必须让执行环境成为可管理对象。

8 Stop、submit 与 validation

长期 agent 必须有停止机制。没有停止机制，agent 可能无限循环；停止太早，可能提交未验证 patch；停止条件太模糊，模型会在“应该继续查”与“应该交付”之间摇摆。

mini-SWE-agent 用 step limit、cost limit、wall-time limit 控制运行，并通过 exit role / sentinel output 表示提交。SWE-agent 有 submit 动作和预算限制。Agentless 则用 pipeline 结束条件：生成候选 patch，运行 regression/reproduction tests，选择最终 patch。

机制	作用	失败形态
step limit	防止无限循环	复杂任务被截断
cost limit	控制推理成本	成功前被迫提交
wall-time limit	控制运行时间	慢测试导致误失败
submit command	明确完成信号	过早提交或忘记提交
validation tests	筛掉坏 patch	测试弱或环境 flaky

validation 是 stop 的另一半。真正的完成不是“模型说完成”，而是交付物通过某种可执行检查。对 coding agent 来说，这通常意味着 patch 能 apply、代码能运行、相关测试通过、旧行为不被破坏。越接近真实工程，validation 越要把当前 issue 和 regression 同时考虑。

8.1 validation 只是局部证明

即使测试通过，也不代表 patch 一定正确。测试只覆盖了某些输入、某些环境、某些行为。coding agent 容易利用这个缺口：写一个让当前测试通过但语义错误的 patch，或者只处理 reproduction case 而破坏 general case。

所以 validation 需要和任务定义绑定。SWE-bench 式 issue repair 关心的是 issue 所描述的真实 bug，而不是任意通过测试；Agentless 使用 regression tests 和 reproduction tests，是为了同时约束“不要破坏旧行为”和“确实修复当前问题”。在 harness 层必须先建立这条原则：验证不是分数装置，而是交付契约。

8.2 停止条件要让模型敢继续，也敢结束

停止策略太严，agent 会在找到根因前提交；停止策略太松，agent 会在低价值探索中消耗预算。一个好的 stop policy 要把预算信号暴露给模型，让模型知道剩余步数和成本；也要提供明确的 submit 动作，让完成不是靠自然语言猜测。

长期任务尤其需要“中间完成感”。例如，定位完成、patch 完成、测试完成、总结风险，都可以作为隐式阶段。否则模型会把所有步骤混在一起：一边改代码，一边猜测试，一边写最终回答，导致轨迹难以审计。

9 Harness families

现在可以把常见 coding-agent harness 分成五类。这不是严格分类，而是帮助读者理解设计空间。

Family	代表	核心思想
Minimal loop	mini-SWE-agent	bash-only、linear history、简单控制流
LM-friendly ACI	SWE-agent	专门为模型设计命令和反馈
Code-as-action	CodeAct	用可执行代码统一工具和数据流
Platform harness	OpenHands	event stream + runtime + UI + skills
Structured pipeline	Agentless	固定 localization/repair/validation 流程

这五类 harness 没有绝对优劣。它们解决的问题不同。

如果目标是研究模型能力，minimal loop 很有价值，因为 scaffold 较少，轨迹透明。

如果目标是提高 repo repair 成功率，ACI 和 validation-heavy pipeline 往往更有效，因为它们减少低层操作错误。

如果目标是开放平台，event stream、runtime API、UI、skills 和 evaluation framework 会变得必要。

如果目标是复杂工具组合，code-as-action 能把 Python 的控制流和包生态转化为 action space。

10 Failure map

harness 失败不一定表现为“模型答错”。它常常发生在 loop 的某个接口上。



失败点	典型表现	应该改哪里
Bad state	模型看不到关键文件或旧观察混乱	context/state management
Bad action	命令格式错、编辑不稳定	action schema / ACI
Bad observation	日志过长、反馈不含原因	observation compression
Bad runtime	依赖装不上、命令超时	sandbox / environment
Bad stop	过早提交、无限循环	stop policy / budget
Weak validation	patch 看似可行但 regression	tests / validation policy

这张图的实践意义是：不要把所有失败都归因到模型。一个 agent 没修好 issue，可能是模型理解差，也可能是 harness 没给到相关文件、编辑接口太脆、测试输出太长、环境不稳定、提交规则不清楚。只有把失败映射回 harness component，才知道下一步该怎么改。

10.1 读一个 agent 系统时的检查表

读任何 coding agent 论文或系统文档时，可以沿着 harness loop 问八个问题。

1. 初始 state 包含什么：issue、tests、repo map、文件、历史，还是只有自然语言任务？
2. 模型每一步输出什么：shell、diff、tool call、Python code、message，还是 pipeline stage 的局部答案？
3. action 在哪里运行：local、Docker、remote sandbox、browser、CI，还是不可执行的离线生成？
4. observation 如何返回：原始日志、结构化结果、局部文件视图、diff、event stream，还是人工摘要？

5. trajectory 是否完整：能否 replay、debug、训练、过滤失败样本？
6. budget 如何控制：step、cost、wall-time、token、并发资源有没有显式限制？
7. submit 如何定义：final answer、patch 文件、git diff、PR、测试报告，还是平台事件？
8. validation 是否充分：只看生成格式，还是运行 reproduction 和 regression checks？

这组问题比“用了哪个模型”更能解释系统差异。很多论文声称 agent 更强，但真正的差异可能来自 action space 更友好、observation 更干净、validation 更严格，或任务分布更简单。反过来，模型分数低也可能不是模型弱，而是 harness 把信息、权限、反馈或预算设计坏了。

Insight. Harness analysis 的目的不是削弱模型贡献，而是把贡献归因做准：模型负责推理和生成，harness 负责把推理接到真实环境。两者缺一不可。

11 最终 takeaway

Agent harness 的基本模式可以压缩成三句话。

- Harness 是把模型输出变成环境动作、再把环境反馈变成模型输入的控制系统。
- Action space、observation、runtime、trajectory、stop 和 validation 共同决定模型能力能否稳定发挥。
- 不同 harness family 代表不同取舍：简单性、LM 友好性、工具组合、平台能力、可控 pipeline。

因此，读任何 coding agent 系统时，不能只问“用了什么模型”。还要问：state 如何构造，actions 如何表示，环境在哪里运行，observation 如何压缩，trajectory 如何保存，什么时候停止，最终 patch 如何验证。回答清楚这些问题，才算真正读懂一个 agent harness。

参考资料

- [1] [SWE-agent](#). agent-computer interface and software-engineering harness
- [2] [mini-SWE-agent](#). minimal bash-only software-engineering agent
- [3] [OpenHands](#). event stream, runtime, platform, and evaluation
- [4] [CodeAct](#). executable code actions for language agents
- [5] [Agentless](#). structured localization-repair-validation pipeline